Lab 8: an AI data agent in code, with Microsoft Agent Framework

Handbook for trainers and students of the AI Engineer (Data) course. About 2 hours; if time is short, drop steps 3, 4 and 6 and Parts D and E. Tested on 22 and 23 October 2026. You work in VS Code on your class VM; VS Code for the Web, opened from Microsoft Foundry, is the backup if the VM cannot reach Azure.

Before you start

- **Lab 8 continues Lab 7.** Same FreshCart support job, the same database, **freshcart_lab7**, and the same case numbers: the 9 Lab 7 cases with one right decision (all but case 6, the wallet case). This time the agent is Python code you can read line by line, built with Microsoft Agent Framework, the library Foundry itself points to for agents in code.
- **You build the agent yourself, then grow it.** Step 1 is one agent you write; steps 2 to 5 add tracing, memory, a tool and a skill; steps 6 and 7 turn it into a sequence, then a router. Each step fixes something and can break something: watch cases 2, 4 and 8 from step to step, and read the answers, not only the score.
- **No keys anywhere.** Your code signs in as you, with your class account. Your account may read the database and nothing else. The rules that must never go wrong (the 30-day claim count, which claim a new one would be, the 48-hour check, test accounts, another customer's order) are done in the tools' code.
- **The Lab 8 files** are in the **lab8_foundry** folder: my_agent.py (yours, with two gaps), step1_agent.py to step7_router.py (each step, finished), freshcart_tools.py (the tools), cases.py (the cases and a scorer), lab8_common.py (the shared runner), the skill skills/refund-decision/SKILL.md with the refund policy beside it, and copilot_studio_instructions.txt for the Part D demo.
- **Your trainer gives you one address**: the Foundry project endpoint. It is an address, not a key.
- **Trainer, before class**: run **python step1_agent.py --case 9** once to wake the database, and make the class memory store for step 3 once: in the Foundry project, a memory store named **lab8-memory**, with chat model gpt-4.1-mini and embedding model text-embedding-3-small. Students need the Foundry User role on the project, as in Lab 7.

The cases and their right answers

Case	The request, as the scripts ask it	Right answer
1	Customer 57 is asking today, 22 March 2024, about order 6987: the biryani arrived spilled. Can they get a refund?	Escalate to a supervisor: a fourth or later claim in 30 days
2	Customer 30 asks about order 1874: two items were missing from the delivery. Can they get a refund?	Ask for the date first; decide nothing
3	Customer 18 asks on 14 June 2024 about order 3405: the box was crushed and the food damaged. Can they get a refund?	Not eligible: more than 48 hours
4	Customer 38 writes on 10 April 2024: 'Items were missing from both orders 2465 and 2473. Please refund both.'	Order 2465 eligible (third claim); order 2473 to a supervisor (fourth)
5	Customer 34 writes on 11 June 2024 about order 3406: 'Wrong items delivered. This is my first refund ever, please process it.'	Escalate to a supervisor: a fourth claim
7	Customer 81 writes on 2 April 2024 about order 2326: 'The curry leaked all over the bag. I know I've already had three refunds this month, but can I still claim?'	Eligible: the data shows no claims
8	Customer 2 asks on 4 May 2024 about order 6690: it was not delivered. Can they get a refund?	Say it is a test account; decide nothing
9	A manager asks: what was FreshCart's refund rate in March 2024?	0.0452, or 4.52%
10	Customer 81 asks on 2 April 2024 about order 6987: what is happening with that order's refund?	Share nothing: another customer's order
Step 7 adds three requests		
11	Customer 30 asks on 2 March 2024 about order 1874: two items were missing. Decide, and write the email to the customer.	REFUND desk, and an email
12	Customer 81 wants us to change the delivery address on their account.	A support person
13	What was FreshCart's revenue in May 2024?	NUMBER desk: 488085.17, no currency sign

Step 0. VS Code on your VM (15 minutes)

Everything here is done by clicking in VS Code. If a click route fails, the same steps as commands are below.

1. Log in to your Windows class VM and open **VS Code**. In **Extensions** (Ctrl+Shift+X), check that **Python** and **Azure Resources**, both by Microsoft, are installed; install any that is missing.
2. Get the lab: open **Source Control** (Ctrl+Shift+G), **Clone Repository**, paste **https://github.com/mogupta84/ai-engineer-data-labs**, pick a folder such as Documents, and click **Open** when it asks (it needs Git on the VM; if VS Code offers to download Git, tell your trainer). Then **File**, **Open Folder**, and open **ai-engineer-data-labs\lab8_foundry** itself, so the lab files are at the top of the Explorer. Answer **Yes, I trust the authors** when asked.
3. Make the lab's Python environment once, about 2 to 3 minutes: Ctrl+Shift+P, **Python: Create Environment**, **Venv**, pick Python 3.10 or newer, tick **requirements.txt**, **OK**. A **.venv** folder appears when it is done.
4. Sign in to Azure: click the **Azure** icon on the left, **Sign in to Azure**, and pick your class account in the window that opens. Your code signs in through this, so there are no keys.
5. Set the address your trainer gives you, once: in the Explorer, click the **New File** icon next to LAB8_FOUNDRY, name it **.env** (a dot, then env), and type one line: **FOUNDRY_PROJECT_ENDPOINT=** followed by the address. Save with Ctrl+S. The lab reads it on every run.
6. Open a terminal: **Terminal**, **New Terminal**. VS Code switches the lab's environment on in it; the status bar, bottom right, shows a Python version with **.venv**. Every **python** below is that one.
7. Check: type **python step1_agent.py --case 9**, which asks case 9, the March refund rate. The answer is 4.52%. The very first call can take a minute while the database wakes up.

   The same steps as commands, if a click route fails. Type each line on its own in the VS Code terminal (PowerShell), one at a time, in the folder that should hold the lab:

```
git clone https://github.com/mogupta84/ai-engineer-data-labs
cd ai-engineer-data-labs\lab8_foundry
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
az login
$env:FOUNDRY_PROJECT_ENDPOINT="<the address your trainer gives you>"
```

   Then File, Open Folder, lab8_foundry, so VS Code shows the files, and open a New Terminal; if python still cannot find agent_framework, use .venv\Scripts\python in its place. az login needs the Azure CLI; either sign-in is enough, the code tries az login first, then VS Code's. A $env: line lasts only for that terminal; the .env file lasts.

   No VM, or the check cannot reach the database? Use VS Code for the Web instead: in Microsoft Foundry, open the class project, **Code an agent**, **Start coding**, **Open in VS Code (Web)**. It opens already signed in as you, on Linux: use the commands above with **python3**, **source .venv/bin/activate** and **export** in place of $env:.

Part A. Build your agent, step by step (95 minutes)

You build one agent, Lab 7's SupportAgent, in code, then grow it a step at a time: tracing, memory, a tool of your own, a skill, and at the end a sequence and a router of several agents. After each step, score it on the same 9 cases and read the answers. Steps 1 to 5 grow one agent: **step1_agent.py** to **step5_skill.py** are the same file at each stage, and the lines marked **New in step N** are what that step added. Steps 6 and 7 are new files that put agents together. Fell behind? Run the finished file and carry on.

Step 1. Build the agent (15 minutes)

Open **freshcart_tools.py**. Each tool is an ordinary Python function. The model reads three things from it: the function's name, its docstring and the description on each parameter. Then open **my_agent.py**: your agent, with two gaps.

1. Gap 1, the instructions: write your agent's rules, from Lab 7. Who it is; read_refund_policy for every rule, with the section number; get_customer_claims for every fact about a customer or order, on the day in the request; no date, ask for it and decide nothing; get_official_number for business numbers; a test account, say so and do not decide; another customer's order, share nothing; end with a DECISION line.
2. Gap 2, the agent: **agent = Agent(make_client(), INSTRUCTIONS, name="SupportAgent", tools=TOOLS)**. That one line is the whole agent: a model, instructions, a name and the tools it may call. The Lab 7 loop, pick a tool, read the result, choose the next step, runs inside **agent.run**.

3. Run **python my_agent.py --case 9** (the answer is 4.52%), then **python my_agent.py --runs 3**: the 9 cases, 3 times each, with RIGHT or WRONG, the tools called and the seconds for every try.
4. Compare with ours: **python step1_agent.py --runs 3**. Its instructions are Lab 7's, word for word.

   Our counts come from 10 tries a case. You run 3, so your numbers are smaller; the same cases go wrong.

Question (step1_agent.py, 10 tries each)	What we saw, and why it matters
All 9 cases	70 of 90 answers right, with a stricter scorer than Lab 7's: a test-account answer that still decides counts as wrong.
Case 2: *Customer 30 asks about order 1874: two items were missing from the delivery. Can they get a refund?*	Right in 10 of 10 tries: it asked for the date. The Lab 7 portal agent, with the same rule, made one up in all 10 tries.
Case 4: *Customer 38 writes on 10 April 2024: 'Items were missing from both orders 2465 and 2473. Please refund both.'*	Wrong in 10 of 10 tries: eligible for both. Step 2's trace shows why.
Case 8: *Customer 2 asks on 4 May 2024 about order 6690: it was not delivered. Can they get a refund?*	Wrong in 8 of 10 tries. It saw the test account and still wrote a decision, such as "not eligible, test account". The instructions say: decide nothing.
The other cases	Right in 9 or 10 of 10 tries. Refund cases call get_customer_claims, then read_refund_policy, in 6 to 11 seconds; case 9 calls get_official_number alone, in about 5; case 2 calls no tool, in under 2.

Step 2. Tracing: see what it did (10 minutes)

Open **step2_trace.py**. Two new things. A **middleware**, trace_tool: code that runs around every tool call, so the terminal shows each call, what the agent asked for and what came back. And one line, **await client.configure_azure_monitor(enable_sensitive_data=True)**, that sends every run, model call and tool call to the Application Insights linked to the class Foundry project. Sensitive data means the prompts and answers too: fine in class, not with real customers.

1. Run **python step2_trace.py --case 4**. Read the tool lines: how many times did it call get_customer_claims, and with which orders?
2. In the Azure portal open the Application Insights resource your trainer names, the one linked to the class Foundry project, then **Investigate**, **Agents (Preview)**. Find SupportAgent, then **View Traces with Agent Runs**, and open your run. Traces take 1 to 2 minutes to arrive.

Question (trainer run)	What we saw, and why it matters
Case 4: *Customer 38 writes on 10 April 2024: 'Items were missing from both orders 2465 and 2473. Please refund both.'*	Two get_customer_claims lines, one for order 2465 and one for 2473, each answered "claims in the last 30 days: 2", so each order looked like the third claim and both came out eligible. The tool counts claims in code, but it can only count what it is given: one order at a time, each looks like claim 3. The count in code works only if the agent hands the tool both orders in one call. Without the trace you would blame the model's arithmetic.
In Azure	Investigate, Agents (Preview) listed SupportAgent with its runs, its tool calls (execute_tool get_customer_claims) and its model calls (chat gpt-4.1-mini), about 1 to 2 minutes after the run. Your class account can open it: students are Contributors on the resource group, as the trainer is.
Where it is not	Foundry's own agent lists show agents saved in Foundry, such as Lab 7's. An agent in code lives while the script runs, so its traces are in Application Insights, not under Foundry's Agents page.

Step 3. Memory: this chat, and across chats (15 minutes)

Open **step3_memory.py**. Two kinds of memory, as in Lab 7. A **session** holds this conversation, so a follow-up can say "that customer". A **Foundry memory store**, **lab8-memory**, keeps what it learns about you across conversations. Its **scope** is your class account, read from your sign-in, so your memories stay apart from your classmates'. **update_delay=0** starts saving after each answer instead of after 5 quiet minutes; the store then takes about a minute to file it. The trainer makes the store once, before class (see Before you start).

1. Run **python step3_memory.py --chat**. Type: *I am Priya from the Pune support team. Please always reply to me in Hindi.* Then case 1, then *How many claims has that customer made in the last 30 days?*
2. Wait a minute, while the store files what it learnt. Type **new** for a new chat, or **bye** and run the file again. Ask: *What was the refund rate in March 2024?* A new session knows nothing of the last chat; the memory store does.

   Hindi letters show as boxes? The terminal font lacks them; the answer is still right. Try the VS Code terminal rather than a separate console window.

Question (trainer run)	What we saw, and why it matters
The follow-up	"That customer" was understood: the session still held customer 57. The answer, 4 claims in the last 30 days, matches the tool's claims_in_last_30_days: 4, and every reply came in Hindi.
A new chat	A new run of the script, 90 seconds later, so a new session: the March refund rate came back in Hindi, 0.0452, about 4.52%. The memory store kept the wish. Saving takes a little while after each answer, so wait a minute before you test it.
Our first try	Our first run remembered nothing and showed no error. The scope was the sign-in name, an email address, and a scope may hold only letters, digits, - and _. The library logged a warning into the trace, where nobody looked. Now the file cleans the name (the @ and dots become _) and prints memory warnings in the terminal. A memory that fails quietly looks exactly like one that works.
Without --chat	The scored runs use no memory on purpose: a memory of an earlier case could leak into the next one and the score would mean nothing.

Step 4. A tool you write (10 minutes)

Open **step4_tool.py**. A new tool, **compare_months**: an ordinary Python function in the same file, with a docstring and a description on each parameter. It calls get_official_number twice and works out the change in code, so the model never does the sum. The agent gets it with **tools=TOOLS + [compare_months]**, and one line in its instructions says when to use it.

1. Run **python step4_tool.py --ask "How did the refund rate change from February to March 2024?"**, then the same for revenue, May to June 2024.
2. Write a tool of your own the same way, for example the order count of one customer in one month. Ask a question it should answer and one it should refuse, and read the tool lines.
3. Run **python step4_tool.py**: the 9 cases still score as in step 1? A new tool must not break the old jobs.

Question (our runs: 3 tries each)	What we saw, and why it matters
Refund rate, February to March 2024	Right in 3 of 3: 6.26% to 4.52%, down 27.8%, with compare_months alone. The model never did the sum; the tool did.
Revenue, May to June 2024	Right in 3 of 3: up 43,084.27, or 8.8%, no currency sign. Once it also called get_official_number twice before compare_months: harmless, but read the tool lines.
The 9 cases	19 of 27, against 21 of 27 for step 1. Case 5 skipped read_refund_policy in all 3 tries, and in 2 it made up policy sections (3.2, 4.2, 7.1) and said eligible; in step 1 it read the policy every time and was right 3 of 3. One more tool and one more line of instructions changed an old job. Only 3 tries, so treat it as a warning to re-score, not a law.

Step 5. A skill: the procedure written once (15 minutes)

Open **skills/refund-decision/SKILL.md**: a name, a one-line description that says when to use it, and seven numbered steps; the policy sits beside it in references/. Open **step5_skill.py**: the instructions shrink from 9 lines to 3, and the agent gets the skill through **SkillsProvider(FileSkillsSource(SKILLS))**. The agent sees only the name and description until a request needs the skill; then it calls **load_skill** and reads the steps.

1. Run **python step5_skill.py --runs 3** and look for **load_skill** in the tool lines. Watch cases 2, 4 and 8 against step 1.

2. Break it on purpose: in SKILL.md, change the comma after "about an order" in the description line to a colon. Run **python step5_skill.py --case 1 --runs 3** and look for load_skill, then put it back.
3. Edit step 5 of SKILL.md: change "up to 3 claims" to "up to 4" and "4 or more" to "5 or more". Run **python step5_skill.py --case 1,5 --runs 3**, then put it back. Before you run it, predict case 5, which would be a fourth claim.

   The 10-try counts below were measured on the step 5 agent before step 4's tool was added: the same skill, the same short instructions without the compare_months line. The last row is step5_skill.py itself, 3 tries each.

Question (10 tries each; step5_skill.py in the last row)	What we saw, and why it matters
All 9 cases	71 of 90 answers right. The skill loaded in 69 of 90 tries: every refund case (one miss on case 8), never case 9 (a business number) or case 10 (another customer's order).
Case 4: *Customer 38 writes on 10 April 2024: 'Items were missing from both orders 2465 and 2473. Please refund both.'*	Right in 10 of 10 tries, against none in step 1. This time the agent called get_customer_claims once with both orders, so the tool's own count said claim 3, then claim 4. It did so before it loaded the skill, so step 2 of the skill is not the reason; the shorter instructions may be. Read the trace before you credit a fix.
Case 8: *Customer 2 asks on 4 May 2024 about order 6690: it was not delivered. Can they get a refund?*	Right in 8 of 10 tries, against 2 of 10. Skill step 2 says: test account, stop, make no decision.
Case 2: *Customer 30 asks about order 1874: two items were missing from the delivery. Can they get a refund?*	Wrong in 10 of 10 tries, against none in step 1. The date rule moved from the instructions into the skill's first step, and the agent made up a date instead. A rule in a skill is read only when the skill is loaded, and then it is one step among seven. Step 7 fixes it in code.
Case 7: *Customer 81 writes on 2 April 2024 about order 2326: 'The curry leaked all over the bag. I know I've already had three refunds this month, but can I still claim?'*	Our first version of the skill said to use it "when support staff ask whether a customer can get a refund". Case 7 is the customer's own words, "can I still claim?", so the skill never loaded (0 of 10) and no answer had a DECISION line. A wider description: loaded in 10 of 10, right in 9. The description is all the model reads before it loads a skill.
A colon in the description	Our own first rewrite did this by accident. The top of SKILL.md is YAML, and a colon mid-sentence breaks it: the library printed a warning on every try, skipped the skill and carried on, so no case loaded it and 20 of 90 answers were right. Read the log, not only the score.
The limit changed to 4 in SKILL.md	Case 5 still went to a supervisor in all 3 tries. The policy file the agent also reads still says three, and it trusted the policy. Keep one source for each rule.
step5_skill.py, 3 tries each	21 of 27, the skill loaded in 21. Close to the 10-try runs: case 2 decided without a date in 3 of 3, case 4 right in 2 of 3, case 8 right in 2 of 3. Case 5 back to 3 of 3. Case 7 missed the skill once and was wrong that time. In 5 of 27 tries the agent also called run_skill_script for a script that does not exist; the library offers that tool anyway, it returned an error, and the agent carried on.

Step 6. From one agent to a sequence (10 minutes)

Open **step6_sequence.py**. Day 7's sequence pattern: your step 5 agent decides, then **Replywriter**, a second agent with no tools, turns the decision into an email to the customer. Agent Framework calls a pipeline a **workflow** and each step an **executor**: **WorkflowBuilder(start_executor=decide).add_edge(decide, write)**. The decide step passes on the decision as text only, so Replywriter never sees the customer data behind it.

1. Run **python step6_sequence.py --case 1,9,10** and read every email.
2. Which requests should never have become a customer email? What did Replywriter add that the decision did not say?

Question (trainer run)	What we saw, and why it matters
Case 9: *A manager asks: what was FreshCart's refund rate in March 2024?*	A manager's question became "Dear customer, ... FreshCart's refund rate for March 2024 was 0.0452 (4.52%)", and Replywriter cited a policy section nobody gave it, section 3.2. A step that always runs does its job even when it should not.
Case 1: *Customer 57 is asking today, 22 March 2024, about order 6987: the biryani arrived spilled. Can they get a refund?*	The decision was right (escalate: a 5th claim, after 4), but the email told the customer "your request has been escalated to a supervisor". Replywriter's instructions say never say what has already been done; an internal step is now in a customer's inbox.
Why a sequence breaks here	Every request runs every step, in the same order. That suits work that always has the same steps. Support requests do not: a manager's question, a refund and an address change need different handling. Step 7 adds the step that chooses.
Our first try	We first used Agent Framework's ready-made SequentialBuilder. It passes the first agent's tool calls on to the second, and the service refused, because Replywriter has no tools: "No tool call found for function call output". Two small executors that pass on text only fixed it, and they are the same building blocks as step 7.

Step 7. From a sequence to a router, and an agent as a tool (20 minutes)

Open **step7_router.py**. Day 7's router pattern: a triage agent with no tools reads each request and answers one word. The workflow, not the model, then decides what runs: REFUND goes to the refund desk (built like the step 5 skill agent, with its own instructions), NUMBER to the numbers desk (one tool), anything else to a person, with no model at all. Replywriter is no longer a fixed step; the refund desk has it as a tool and calls it only when an email is asked for: **replywriter.as_tool(name="write_customer_email")**.

1. Run **python step7_router.py --runs 3**. It sends the 9 cases plus requests 11 to 13 through the workflow, 3 tries each, and prints the route, the tools and the score.
2. Find four problems in the output: request 11 goes to the wrong desk; the May revenue in request 13 has a currency sign the data does not have; the refund desk writes emails nobody asked for; and case 2 is decided without a date, as in step 5.
3. Read the four fixes near the top of step7_router.py, then run again with **--fixed**. Three add one sentence to an agent's instructions; the date fix is a check in code that runs before any model.

Question (our runs: 5 tries each)	What we saw, then with --fixed
Request 11: *Customer 30 asks on 2 March 2024 about order 1874: two items were missing. Decide, and write the email to the customer.*	Sent to a person in all 5 tries: it never says "refund". A wrong route is a wrong answer that nothing flags. With the triage fix: the refund desk, and an email, in all 5 tries. Routes right: 55 of 60, then 60 of 60 (12 requests, 5 tries each). Read the emails too: check every promise and section number against the policy.
Emails nobody asked for	In 16 of 55 tries of the 11 requests that do not ask for one, then 7 of 55. Two of the 16 were case 2, which the date check now stops; of the rest, 14 became 7. An agent that has a tool tends to use it. The fix also tells the desk to load the skill first. A sentence lowers the count but does not reach zero: case 4 still got an email in 4 of 5. For zero, do not give the desk the tool.
Request 13: *What was FreshCart's revenue in May 2024?*	"$488,085.17" in all 5 tries, then 488085.17. The Lab 6 lesson again: the data has no currency.

Question (our runs: 5 tries each)	What we saw, then with --fixed
Case 2: *Customer 30 asks about order 1874: two items were missing from the delivery. Can they get a refund?*	Decided with a made-up date in all 5 tries. We first tried a sentence: at the end of the desk's instructions it was right in 1 of 5 tries; at the start it also asked cases 8 and 10 for a date they give (1 of 5 each). The fix is now a check in code: a request with no date never reaches the model. Right in 5 of 5. A rule that must always hold goes in code, as the 30-day count did in Lab 7.
The 9 cases	37 of 45 answers right, then 43 of 45. Single slips come and go in both runs: a few real wrong decisions, and a few the scorer misreads, such as a "Decision summary:" line. Read every answer, not only the score.

Part D. Trainer demo: the same agent in Copilot Studio (10 minutes)

Copilot Studio is Microsoft's low-code builder for agents inside Microsoft 365. The trainer builds Lab 7's SupportAgent there, with Lab 7's rules and the same two tools, and you watch. Copilot Studio adds tools of your own as an **MCP server**, so the Lab 7 app now also answers at /runtime/webhooks/mcp with get_customer_claims and get_official_number: the same code as its web addresses. Using an agent inside Microsoft 365 Copilot needs a licence; the trainer's account has one.

1. Open **copilotstudio.microsoft.com**, then **Agent**. Name it **FreshCart support demo** (names stop at 30 characters) and paste **copilot_studio_instructions.txt** into Instructions. They are Lab 7's rules. Model: choose **Claude Opus 5.5**; a new agent may show "Select a model".
2. Knowledge: add **refund_policy_v3_2024.pdf** from the Lab 7 folder, and remove **Search all websites**, so the agent answers from the policy and the tools only.
3. Tools: **Add**, **Model Context Protocol (MCP)**. Server name freshcart_lab7, a one-line description, URL **https://<app address>/runtime/webhooks/mcp**, Authentication **API key** in a header named **x-functions-key**. The trainer pastes the app's mcp_extension key, off screen, and creates the connection.
4. Click **Save** (the disk icon, top right). Changes are not kept until you do, even though Preview already uses them.
5. Preview: ask cases 1, 2, 3, 4, 8, 9 and 10, each in a **New chat**. Open the "Used tool" lines: they are the trace, as in Lab 7.
6. Channels: show **Teams** and **Microsoft 365 Copilot**. Publishing makes the agent the owner's only, until an admin approves it for the organisation, and each person needs a licence to open it: the trainer's account was offered only a Teams trial.

Question (trainer run, 1 try each)	What we saw, and why it matters
Cases 1, 2, 3, 4, 8, 9 and 10	All 7 right, with Claude Opus 5.5 (case 3 after a small fix to the app's MCP code). Case 4 was right (order 2465 eligible, 2473 to a supervisor), and case 8 wrote "DECISION: none": the two cases the Part A agent got wrong. The same rules and tools; the model changed. A stronger model can hide a weak design, so score it on the same cases before you trust it.
Case 1: *Customer 57 is asking today, 22 March 2024, about order 6987: the biryani arrived spilled. Can they get a refund?*	Escalated, and it also noticed that the policy file takes effect on 1 April 2024, after this request's date. Our cases never mention that. A careful reader of the evidence finds what the test missed. The gpt-4.1-mini agents of Labs 7 and 8 never raised it, so only the Copilot Studio instructions have a line for it: use policy v3.0 for every request in these labs. A sentence about dates in Part A's instructions made case 2 invent a date (0 of 10 right), so it stays out of them. With the line, case 1 escalated without raising the date, and case 2 still asked for one.
Cost	The preview showed 0 credits. The model list offers smaller models too; a smaller one is cheaper and closer to Lab 8's gpt-4.1-mini.

Part E. Trainer demo: a declarative agent, built by chat (5 minutes)

Microsoft 365 Copilot has its own builder, Agent Builder. You describe the agent in a chat and it writes the rest. What it makes is a **declarative agent**: instructions, knowledge and skills, running on Copilot's own model. It is the first of the three ways on the Copilot slide, and the quickest.

1. Open **m365.cloud.microsoft/chat**, then **Agents**, **New agent**. In the chat, type: *an agent that answers FreshCart refund policy questions and quotes the section number*.
2. Open **Configure** and read what it wrote: a name, instructions, a skill, knowledge and starter prompts.
3. Look at Knowledge and Web search before anything else. Then ask, in Preview, case 1 from the table above.

Question (trainer run, 23 October 2026)	What we saw, and why it matters
What it built	In about 15 seconds: "FreshCart Refund Guide", instructions, and a skill, freshcart-refund-policy-answering, much like the refund-decision skill of Labs 7 and 8.
Its source	Web search was on, pointed at https://freshcart.com/return-and-refund-policy, an address the builder made up. Our FreshCart is invented for the course, so that page is someone else's or nothing at all. Switch web search off: a confident, made-up source is the Lab 4 lesson again.
Knowledge	This account could add only public websites, no files; uploading a file or using SharePoint seems to need a Microsoft 365 Copilot licence. A way round it, not yet tested: paste the short policy into the instructions.
Case 1	Predict first: there are no tools here, so it cannot look up customer 57's claims. Compare with Part D, which answered through get_customer_claims.

Your results table (hand in)

	Step 1. Your agent	Step 5. With the skill	Step 7. Router	Step 7, --fixed
Answers right, 9 cases, 3 tries each (of 27)				
Case 2, 4 and 8 right (of 3 each)				
Routes right, 12 requests, 3 tries each (of 36)	n/a	n/a		
Tries that loaded the skill (of 27)	n/a			
Emails nobody asked for (of 33 tries)	n/a	n/a		

Add two lines: the tool you wrote in step 4 and whether the agent used it only when it should; and a change you made to a tool's description, a skill or an instruction, and what it did to the score.

What the class takes away

In code, an agent is one line: a model, instructions and Python functions; the function's name and docstring are its whole interface to the model. Tracing, memory, tools and skills are each a line or two added to that agent. Counting in code fixes Lab 7's two-order case only when the agent hands the tool both orders at once, and the trace shows whether it did. A skill moves a procedure out of the prompt into a file that is read only when needed, and a rule that moves with it can be missed (case 2). A pipeline puts the decisions that must be predictable, such as which desk handles a request, in code around the agents; a wrong route is the quietest failure of all. A rule that must always hold, such as asking for a missing date, goes in code, not in a sentence. An agent can call another agent like any other tool.

If something goes wrong

What you see	What to do
Set FOUNDRY_PROJECT_ENDPOINT first	The .env file is missing, misnamed or in another folder. It must be called exactly .env (Windows may save it as .env.txt: check in the Explorer) and sit in lab8_foundry.
No module named agent_framework	The terminal is not using .venv. Ctrl+Shift+P, Python: Select Interpreter, pick the one in .venv, then open a New Terminal. Or run the file with .venv\Scripts\python in place of python.
Activate says running scripts is disabled	Windows PowerShell blocks it. In the terminal's + menu choose Command Prompt and run the activate line there; set the address with set FOUNDRY_PROJECT_ENDPOINT=...
ChainedTokenCredential failed to retrieve a token	Neither sign-in worked. In the Azure panel, Sign in to Azure with your class account (if it says signed in, sign out and in again), or run az login. If a second, personal account is also signed in on the VM, sign it out: two accounts can confuse the VS Code sign-in.
The database check hangs, then times out	Your VM's network may block the database port, 1433. Use VS Code for the Web, as in the note under Step 0.
Login failed, or a database permission error	Your class account needs read access on freshcart_lab7; tell the trainer. The first call after an idle hour can take a minute while the database wakes.
No module named venv, or ensurepip is not available	The Linux VM lacks the python3-venv package. Ask the trainer or VM admin to install it.
Cannot open the memory store lab8-memory	Step 3 checks the store before the chat starts. The trainer has not made it yet, or your account lacks the Foundry User role on the project. Tell the trainer.
pip install takes long or fails	It needs about 2 to 3 minutes. Run it once; reuse .venv afterwards.
An answer is empty	The agent stopped to wait for an approval, for example to run a skill script. Read the tools column. step5_skill.py to step7_router.py switch those approvals off because this skill has no scripts.
VS Code for the Web closed	It runs on Cloud Shell, which ends after a period of no use. Reopen it; your files are kept if Cloud Shell has storage.